CreditSense v2

Milestone 2 - Professional Visual Report

AI-Assisted Credit Risk Assessment with RAG, LangGraph, Policy Engine, and Bilingual Reporting

Purpose of this report: This is a viva-ready technical document that explains architecture, code flow, design decisions, challenges, and proof of ownership in a highly visual and readable format.

Repository:	GEN_AI_LOAN_APPROVAL
Milestone Focus:	MILESTONE 2 / creditsense
Stack:	Streamlit, FastAPI, LangGraph, ChromaDB, ONNX MiniLM (via Chroma)
Report Version:	As-built from code scan on 2026-04-19

This PDF is intentionally designed to be understandable for viva panel members who may not read source code line-by-line.

Contents

1 Project Snapshot **2**

1.1 What We Built . . . 2

1.2 Delivered Milestone 2 Scope . . . 2

2 Viva Voce Preparation **2**

2.1 System Architecture (The How) . . . 2

2.2 Project Architecture Diagram . . . 2

2.3 Turn Lifecycle Diagram (Input to Output Pipeline) . . . 3

2.4 LangGraph Control Flow Diagram . . . 4

2.5 Architecture Diagram Upload (for Viva Portals) . . . 4

3 Code Understanding Guide (Read This Before Viva) **5**

3.1 How to Navigate the Codebase Fast . . . 5

3.2 Code Snippet 1: Graph Routing Logic . . . 5

3.3 Code Snippet 2: API Seed Parameter Normalization . . . 5

3.4 Code Snippet 3: Deterministic Policy Scoring . . . 6

3.5 Code Snippet 4: Retrieval Query Composition . . . 6

4 Challenges Faced **7**

5 Tool Deep-Dive **7**

6 Design Reasoning & Justification **8**

6.1 Why this specific architecture . . . 8

6.2 Why this model and retrieval stack . . . 8

6.3 Why hybrid deterministic + ML + LLM . . . 8

7 Learnings / Key Takeaways **9**

8 Code Ownership & Integrity Declaration **9**

9 Rapid Viva Q&A Drill **9**

A Appendix A: Exact Local Run Commands **10**

B Appendix B: Mermaid Diagram Export Commands **10**

1 Project Snapshot

1.1 What We Built

CreditSense v2 is a conversational credit assessment system where a user enters borrower details, interacts with an AI assistant, and receives a structured decision report supported by regulatory citations from an indexed RAG corpus.

1.2 Delivered Milestone 2 Scope

Area	Implemented Capability
Frontend	Streamlit UI for intake form, chat, decisions, and downloadable reports
Backend	FastAPI endpoints for health, state initialization, turn processing, seeded parameters, and ingestion
Orchestration	LangGraph node graph with deterministic routing
Retrieval	Chroma persistent store + ONNX MiniLM embeddings (via Chroma)
Decisioning	Deterministic policy thresholds + ML adapter score + risk flags
Reporting	English report generation + Hindi translation + PDF export
Deployment Assets	Procfile, render.yaml, runtime.txt (Python 3.11.0)

2 Viva Voce Preparation

2.1 System Architecture (The How)

The architecture is intentionally layered so each responsibility is clear and auditable:

1. Presentation Layer: Streamlit UI captures borrower profile and user questions.
2. API Layer: FastAPI validates payloads, normalizes inputs, and invokes graph turns.
3. Orchestration Layer: LangGraph runs deterministic node transitions.
4. Intelligence Layer: Retriever + policy + ML + LLM compose final analytical output.
5. Output Layer: Bilingual report rendering and PDF export.

2.2 Project Architecture Diagram

Instruction alignment: architecture is provided as a built-in visual figure in this report and also as Mermaid source for external export/upload.

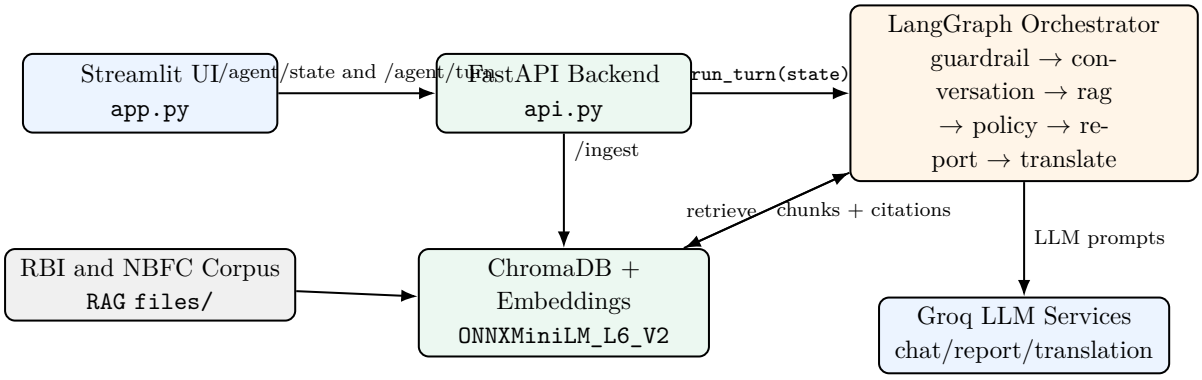

Figure 1: CreditSense v2 component architecture (rendered directly in LaTeX)

2.3 Turn Lifecycle Diagram (Input to Output Pipeline)

Figure 2: End-to-end data flow from user input to final downloadable output

2.4 LangGraph Control Flow Diagram

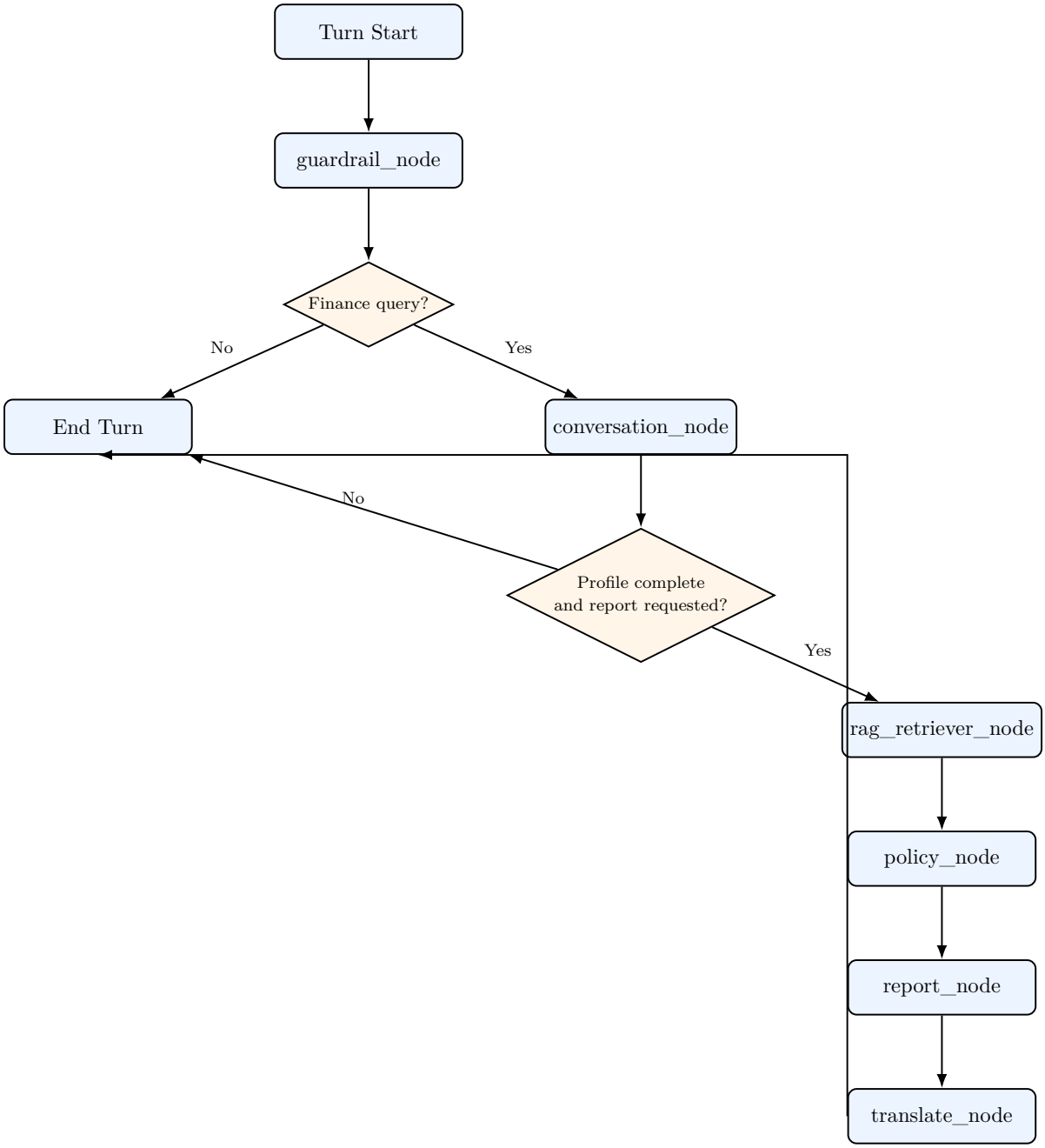

Figure 3: Deterministic control logic implemented in `agent/graph.py`

2.5 Architecture Diagram Upload (for Viva Portals)

If your form asks for *Project Architecture Diagram* image upload:

1. Use Mermaid source: `docs/diagrams/system-architecture.mmd`
2. Export PNG via Mermaid CLI
3. Upload `docs/diagrams/exports/system-architecture.png`

3 Code Understanding Guide (Read This Before Viva)

3.1 How to Navigate the Codebase Fast

File / Module	Why It Matters
api.py	API contract and entry point for all runtime actions
agent/graph.py	Actual node graph and routing conditions
agent/nodes.py	Business logic inside each graph node
services/policy_engine.py	Deterministic decision scoring and risk flags
services/ml_adapter.py	ML model loading + fallback heuristic scoring
services/retriever.py	Query building and Chroma retrieval
services/report_generator.py	English report composition with fallback
services/translator.py	Hindi translation behavior

3.2 Code Snippet 1: Graph Routing Logic

```
def _after_conversation(state: AgentState) -> str:
    return (
        "rag_retriever"
        if state.get("profile_complete", False)
        and state.get("report_requested", False)
        else END
    )

graph.add_conditional_edges(
    "conversation",
    _after_conversation,
    {
        "rag_retriever": "rag_retriever",
        END: END,
    },
)
```

Listing 1: Conditional routing in graph orchestration

Why this matters:

- Prevents report generation on incomplete borrower profile.
- Makes decision path deterministic and explainable to reviewers.

3.3 Code Snippet 2: API Seed Parameter Normalization

```
def _normalize_seed_parameters(parameters: dict[str, Any]) -> dict[str,
    Any]:
    normalized: dict[str, Any] = {}
    for field in REQUIRED_FIELDS:
        if field not in parameters:
            continue
        value = parameters.get(field)
```

```
        if value is None:
            continue
        if isinstance(value, str):
            value = value.strip()
            if not value:
                continue
        if field in INT_FIELDS:
            normalized[field] = int(float(value))
            continue
        if field in FLOAT_FIELDS:
            normalized[field] = float(value)
            continue
        normalized[field] = str(value)
    return normalized
```

Listing 2: Strong typing and normalization before scoring

Why this matters:

- Prevents malformed user input from corrupting downstream policy and ML logic.
- Ensures consistent numeric operations for DTI, EMI, and LTV calculations.

3.4 Code Snippet 3: Deterministic Policy Scoring

```
score = 100.0
for check in checks:
    status = check["status"]
    if status == "FAIL":
        score -= 18
        risk_flags.append(f"{check['metric']} failed")
    elif status == "WATCH":
        score -= 10
        risk_flags.append(f"{check['metric']} needs review")

if score >= 80:
    decision = "APPROVE"
elif score >= 60:
    decision = "CONDITIONAL"
else:
    decision = "ESCALATE"
```

Listing 3: Transparent score deductions based on check status

Decision formula used in viva explanation:

$$S = \max\left(0, \min\left(100, 100 - 18F - 10W\right)\right)$$

where F is number of `FAIL` checks and W is number of `WATCH` checks.

3.5 Code Snippet 4: Retrieval Query Composition

```python
def build_query_from_profile(profile, ratios, conversation_context="")
    -> str:
    return " ".join([
        f"loan purpose {profile.get('loan_purpose', '')}",
        f"loan amount {profile.get('loan_amount_requested', 0)}",
        f"credit score {profile.get('credit_score', 0)}",
        f"post loan dti {ratios.get('post_loan_dti', 'na')}",
        f"collateral {profile.get('collateral_type', 'none')}",
        f"borrower conversation context {conversation_context[:400]}",
        "RBI NBFC underwriting fair practices",
    ])
```

Listing 4: Profile-aware retrieval query for regulatory grounding

Why this matters:

- Retrieval is not generic; it is conditioned by borrower profile and financial ratios.
- The response can reference relevant regulatory context instead of unsupported text.

4 Challenges Faced

Challenge	Impact Risk	Resolution Implemented
Incomplete borrower intake	Wrong or premature decisions	Introduced missing-field tracking and conditional routing before report path.
LLM downtime or unstable responses	Broken UX and inconsistent outputs	Implemented deterministic fallback for chat guidance, report generation, and translation.
State field loss during graph execution	Conversation corruption	Added explicit full-state merge after graph invocation.
Port conflicts in local runs	Startup friction during demo	Startup scripts now clear stale listeners and enforce known defaults.
Hindi PDF rendering gaps	Unreadable export artifacts	Configurable Devanagari font path with fallback messaging in exporter.

5 Tool Deep-Dive

Library	**Role**	**Deep Project Understanding**
Streamlit	Frontend runtime	Used for high-speed form + chat UX with session state continuity and integrated report rendering/-download controls.

FastAPI + Pydantic	API and contract layer	Strong schema validation reduces malformed payloads and keeps state transitions predictable.
LangGraph	Workflow orchestration	Provides deterministic graph control so every report path is auditable and viva-explainable.
ChromaDB	Vector retrieval store	Persistent local retrieval is used for regulatory chunk lookup and citation generation.
ONNX MiniLM via Chroma	Embedding generation	ONNXMiniLM_L6_V2 is used in ingestion and query retrieval without a separate sentence-transformers dependency.
scikit-learn + joblib	ML scoring adapter	Loads trained model when available, otherwise falls back to deterministic heuristic scoring.
FPDF2	PDF exporter	Converts final report text into portable output; supports multilingual path with custom font registration.
python-dotenv	Configuration bootstrap	Ensures env vars are loaded before settings are consumed by services.
httpx + Groq integration	LLM access layer	Handles model calls while enabling key rotation and graceful fallback behavior.

6 Design Reasoning & Justification

6.1 Why this specific architecture

- Regulated credit decisions require explainability; deterministic policy + graph routing was mandatory.
- User trust requires readable narrative; LLM is used for explanations, not sole decision authority.
- Milestone constraints favored rapid deployment and maintainability; Streamlit + FastAPI gave fastest reliable path.

6.2 Why this model and retrieval stack

- ONNX MiniLM via Chroma chosen for practical retrieval quality under CPU-focused runtime constraints.
- Chroma chosen for local persistence and simple operational footprint.
- Groq-backed large model chosen for report language quality and Hindi translation quality.

6.3 Why hybrid deterministic + ML + LLM

A single-method system is brittle:

- Pure deterministic: strong control, weak narrative intelligence.

- Pure LLM: strong language, weak auditability.
- Hybrid approach: preserves compliance explainability while keeping conversation/report quality high.

7 Learnings / Key Takeaways

1. Architecture clarity matters more than model size in viva evaluations.
2. Deterministic fallbacks are essential for production-like reliability.
3. Good retrieval requires query engineering, not only vector DB setup.
4. Clear state contracts simplify debugging in conversational AI systems.
5. Visual documentation dramatically improves reviewer comprehension.

8 Code Ownership & Integrity Declaration

By checking these items, we certify viva readiness:

- ☐ **Code:** I can answer specific questions about any part of my own code.
- ☐ **Ownership:** I can provide clear proof that I/we performed the work.

Recommended viva proof bundle:

- Git history and commit timeline.
- Live walkthrough of endpoint call path and graph execution.
- Demonstration of report generation from fresh seeded borrower profile.

9 Rapid Viva Q&A Drill

Q1: Explain data flow from input to output in 30 seconds.
A: User enters profile in Streamlit, API normalizes and merges state, LangGraph runs guardrail and conversation. If profile is complete and report requested, retrieval fetches regulatory evidence, policy computes decision score, report and translation nodes generate bilingual outputs, and PDF export completes delivery.

Q2: Why is your decisioning defensible?
A: Decision score is deterministic with explicit deductions per failed/watch checks, and each decision band is transparent (APPROVE, CONDITIONAL, ESCALATE).

Q3: What happens when LLM API fails?
A: The system remains functional using deterministic fallback for chat, report generation, and translation.

Q4: How do you prove regulatory grounding?
A: Retriever pulls citation-ready chunks from indexed RBI/NBFC corpus conditioned by borrower profile and conversation context.

Q5: If you had one week more, what would you improve first?
A: Add authentication, observability traces, and external persistent vector storage for production hardening.

A Appendix A: Exact Local Run Commands

```
cd MILESTONE 2/creditsense
pip install -r requirements.txt
python3 scripts/ingest.py --source-dir "../../RAG files"
bash scripts/run_backend.sh
# second terminal
bash scripts/run_streamlit.sh
```

B Appendix B: Mermaid Diagram Export Commands

```
npm install -g @mermaid-js/mermaid-cli
mmdc -i docs/diagrams/system-architecture.mmd -o docs/diagrams/exports/
   system-architecture.png
mmdc -i docs/diagrams/dataflow-pipeline.mmd -o docs/diagrams/exports/
   dataflow-pipeline.png
mmdc -i docs/diagrams/langgraph-control-flow.mmd -o docs/diagrams/
   exports/langgraph-control-flow.png
mmdc -i docs/diagrams/deployment-view.mmd -o docs/diagrams/exports/
   deployment-view.png
```